# laborbericht.

**Biosignalverarbeitung Laborbericht (BA-MGST-B-3-BB-BS-ILV)**

**Aufbau der Hardware & Datenakquise**

**Labor 1 – Gruppe X**


Medizin- Sport- und Gesundheitstechnik

3. Semester

Lehrveranstaltungsleiter: Aitor Morillo

Jahrgang: MGST-25

Verfasser: Matthias Zoller, Laurenz Keller, Matteo Harrasser


27. 10. 2026

# inhaltsverzeichnis.

**1 Einleitung** **3**

**2 Aufbau des Messsystems (Aufgabe 1)** **3**

**3 Vorausgehende Arbeiten und Berechnungen** **4**

3.1 Umrechnung eines ADC-Wertes in eine Temperatur (Aufgabe 2) . . . . . 4

3.2 Achsenzuordnung und Umrechnung in physikalische Größen (Aufgabe 3) . . . . . 4

**4 Durchführung und Ergebnisse** **5**

4.1 Auf- und Abbewegung mit dem Originalcode (Aufgabe 4) . . . . . 5

4.2 Anpassung der Sensorkonfiguration (Aufgabe 5) . . . . . 5

4.3 Zeitstempel und statische Messung in sechs Orientierungen (Aufgabe 6) . . . . . 6

4.4 Filterung der Daten (Aufgabe 7) . . . . . 7

4.5 Bestimmung der Abtastfrequenz (Aufgabe 8) . . . . . 7

4.6 Mobiles Messsystem (Aufgabe 9) . . . . . 8

**5 Zusammenfassung** **8**

**A Anhang: Abgegebene Dateien (Aufgabe 10)** **9**

# abbildungsverzeichnis.

1 Blockschaltbild des stationären Messsystems mit Komponenten, Bussen und Signalpfaden. . . . . 3

2 Beschleunigung während der Auf- und Abbewegung mit dem Originalcode (Ausschnitt 4 s). . . . . 5

3 Beschleunigung während der Auf- und Abbewegung mit angepasstem Messbereich und Datenrate (Ausschnitt 4 s). . . . . 6

4 Statische Messung in allen sechs Orientierungen des Sensors (je 10 s). . . . . 7

5 Vergleich der ungefilterten und mit einem Butterworth-Tiefpass gefilterten Beschleunigungsdaten. . . . . 7

6 Verteilung der Abtastintervalle des stationären Messsystems. . . . . 8

# tabellenverzeichnis.

1 Relevante Register des MMA8452Q und mögliche Parameterwerte. . . . . 6

2 Abtastfrequenz des stationären (USB) und des mobilen (DataLogger) Messsystems. 8

# 1 einleitung

Ziel des ersten Labors ist der Aufbau eines Messsystems zur Erfassung von Beschleunigungsdaten sowie deren Verarbeitung in Python. Dazu wird ein 3-Achsen-Beschleunigungssensor über $I^2C$ an einen Mikrocontroller angebunden, die Daten über die serielle Schnittstelle an einen PC übertragen und anschließend eingelesen, gefiltert und dargestellt. Untersucht werden insbesondere die Auswirkung der Sensorkonfiguration (Messbereich und Datenrate) auf die Qualität der Messdaten, die tatsächlich erreichte Abtastfrequenz sowie der Unterschied zwischen stationärem (USB) und mobilem (DataLogger, Batterie) Messsystem.

Der zugehörige Python-Code (`MainCode1.ipynb`) bearbeitet die Aufgaben 4 (a), 5, 6 (a), 7, 8, 9 und 10 und ist zusammen mit allen Datensätzen und dem modifizierten Arduino-Code der Abgabe beigelegt.

# 2 aufbau des messsystems (aufgabe 1)

Abbildung 1 zeigt das stationäre Messsystem mit allen Komponenten, Bussen und Signalpfaden.

[**TODO:** Sensortyp prüfen! Im Blockschaltbild steht *BMA400*, der vorgegebene Code `Lab1Code1` verwendet aber die Bibliothek `SparkFun_MMA8452Q` (Sensor MMA8452Q, Default `ODR_1` = 1,56 Hz statt 100 Hz). Abbildung und Text an den tatsächlich verwendeten Sensor anpassen. Die Aufgaben 3 und 5 unten sind für den MMA8452Q geschrieben. Zusätzlich fehlt im Bild die Taktfrequenz des ATmega328P (16 MHz).]

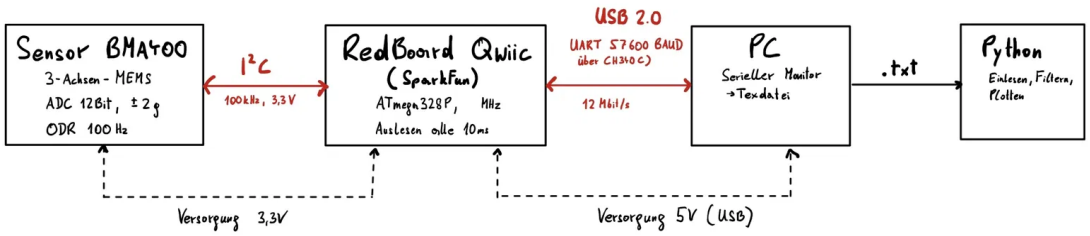



Abbildung 1: Blockschaltbild des stationären Messsystems mit Komponenten, Bussen und Signalpfaden.

## KOMPONENTEN

**Beschleunigungssensor:** Digitaler 3-Achsen-MEMS-Beschleunigungssensor, der die Beschleunigung intern mit 12 Bit digitalisiert (Messbereich einstellbar, Standard $\pm 2\,g$). Die Werte werden mit der eingestellten Datenrate (ODR) in den Datenregistern bereitgestellt.

**SparkFun RedBoard Qwiic:** Arduino-Uno-kompatibles Board mit dem Mikrocontroller ATmega328P (16 MHz), der den Sensor konfiguriert, die Rohdaten ausliest und jeden Messwert mit einem Zeitstempel (`millis()`) versieht. Ein USB-Seriell-Wandler (CH340C) leitet die Daten an den PC weiter.

**PC (Serieller Monitor):** Der serielle Monitor der Arduino IDE empfängt die Messwerte zeilenweise als Text (`X Y Z Zeit`) und zeigt sie an. Die Daten werden anschließend per Copy & Paste in einer Textdatei gespeichert.

**Python:** Die Textdatei wird mit `pandas` eingelesen, mit `SciPy` gefiltert und mit `matplotlib` dargestellt und ausgewertet.

### BUSSE UND SIGNALPFADE

**I²C (100 kHz, 3,3 V):** Serieller Zweidraht-Bus (SDA/SCL), über den das RedBoard den Sensor konfiguriert und die Messwerte ausliest.

**USB 2.0 Full Speed (12 Mbit/s):** Überträgt die Messwerte vom RedBoard zum PC; die eigentliche Datenrate ist durch die UART-Verbindung zum CH340C auf 57 600 Baud begrenzt.

**Datenpfad PC → Python:** Die Messdaten werden als Textdatei (`.txt`) an Python übergeben.

**Spannungsversorgung:** Das RedBoard wird über USB mit 5 V versorgt und versorgt den Sensor über einen Spannungsregler mit 3,3 V.

# 3 vorausgehende arbeiten und berechnungen

## 3.1 UMRECHNUNG EINES ADC-WERTES IN EINE TEMPERATUR (AUFGABE 2)

Gegeben sind ein Temperatursensor mit Ausgangsspannung $U = 0 \ldots 3{,}3\,\mathrm{V}$ für $\vartheta = 0 \ldots 40\,^\circ\mathrm{C}$ (linear) und ein Mikrocontroller-ADC mit $n = 10\,\mathrm{Bit}$ und Referenzspannung $U_{\mathrm{ref}} = 5\,\mathrm{V}$. Der ADC liefert den Wert $D = 520$.

**Schritt 1 – ADC-Wert in Spannung:** Mit $2^{10} - 1 = 1023$ als größtem Digitalwert gilt

$$U = \frac{D}{2^n - 1} \cdot U_{\mathrm{ref}} = \frac{520}{1023} \cdot 5\,\mathrm{V} \approx 2{,}542\,\mathrm{V}. \tag{1}$$

**Schritt 2 – Spannung in Temperatur:** Die Sensorkennlinie hat die Empfindlichkeit $\frac{40\,^\circ\mathrm{C}}{3{,}3\,\mathrm{V}} \approx 12{,}12\,^\circ\mathrm{C/V}$:

$$\vartheta = \frac{U}{3{,}3\,\mathrm{V}} \cdot 40\,^\circ\mathrm{C} = \frac{2{,}542\,\mathrm{V}}{3{,}3\,\mathrm{V}} \cdot 40\,^\circ\mathrm{C} \approx \mathbf{30{,}8\,^\circ C}. \tag{2}$$

Rechnet man mit $2^{10} = 1024$ Stufen, ergibt sich $\vartheta \approx 30{,}78\,^\circ\mathrm{C}$; der Unterschied ist vernachlässigbar. Da der Sensor maximal 3,3 V ausgibt, werden nur $\frac{3{,}3}{5} \cdot 1023 \approx 675$ der 1024 Stufen genutzt; die effektive Auflösung beträgt daher $\frac{40\,^\circ\mathrm{C}}{675} \approx 0{,}06\,^\circ\mathrm{C}$ pro Stufe. [**TODO:** Optional: kurzer Satz, warum 30,8 °C unter der Körperkerntemperatur liegt (z. B. Hauttemperatur).]

## 3.2 ACHSENZUORDNUNG UND UMRECHNUNG IN PHYSIKALISCHE GRÖßEN (AUFGABE 3)

**Achsenzuordnung:** Der Arduino-Code gibt pro Zeile die Werte in der Reihenfolge `getX()`, `getY()`, `getZ()` und anschließend den Zeitstempel aus. Die erste Spalte entspricht somit der $x$-Achse, die zweite der $y$-Achse und die dritte der $z$-Achse des auf der Platine aufgedruckten Koordinatensystems.

**Test:** Im Ruhezustand misst der Sensor nur die Erdbeschleunigung. Legt man den Sensor flach auf den Tisch (z-Achse nach oben), muss die dritte Spalte ca. $+1\,g$ und die anderen beiden ca. $0\,g$ anzeigen. Kippt man den Sensor nacheinander so, dass die $x$- bzw. $y$-Achse nach oben zeigt, muss jeweils die entsprechende Spalte auf $+1\,g$ springen (nach unten: $-1\,g$). [**TODO:** Eigene Beobachtung beim Test kurz beschreiben (welche Spalte hat reagiert).]

**Einheit:** `getX()` usw. liefern den *Rohwert* des 12-Bit-Datenregisters als vorzeichenbehaftete ganze Zahl (Zweierkomplement, $-2048 \ldots +2047$) in Digits (LSB), also noch keine physikalische Größe.

**Umrechnung:** Laut Datenblatt [1] beträgt die Empfindlichkeit $S = 1024\,\mathrm{LSB}/g$ (Messbereich $\pm 2\,g$), $512\,\mathrm{LSB}/g$ ($\pm 4\,g$) bzw. $256\,\mathrm{LSB}/g$ ($\pm 8\,g$) [**TODO:** Seite/Tabelle im Datenblatt eintragen]. Damit gilt

$$a\,[\mathrm{m/s^2}] = \frac{D_{\mathrm{roh}}}{S} \cdot g_0, \qquad g_0 = 9{,}81\,\mathrm{m/s^2}. \tag{3}$$

Beispiel: Ein Rohwert von 1024 bei $\pm 2\,g$ entspricht $1\,g = 9{,}81\,\mathrm{m/s^2}$.

# 4 durchführung und ergebnisse

## 4.1 AUF- UND ABBEWEGUNG MIT DEM ORIGINALCODE (AUFGABE 4)

Der Sensor wurde in der geschlossenen Hand gehalten und 10 s lang einmal pro Sekunde schnell auf und ab bewegt (ohne Rotation). Die Daten wurden mit dem unveränderten Code `Lab1Code1` (`SCALE_2G`, `ODR_1`) aufgezeichnet und per Copy & Paste in eine `.txt`-Datei übertragen.

**(a)** Abbildung 2 zeigt einen Ausschnitt von 4 s der Messung.

**PLATZHALTER – Abbildung noch einfügen**

Aufgabe 4(a): 4 s der Rohdaten (x, y, z) über der Zeit, Originalcode. Achsen: $t$/s und $a/g$; Kurven mit unterschiedlichem Linienstil + Legende.

Abbildung 2: Beschleunigung während der Auf- und Abbewegung mit dem Originalcode (Ausschnitt 4 s).

**(b) Plateaus der Peaks:** Die Peaks enden alle beim selben Wert, weil der Sensor in die *Sättigung* geht: Bei der Einstellung $\pm 2\,g$ kann das 12-Bit-Register maximal +2047 bzw. −2048 LSB ($\approx \pm 2\,g$) darstellen. Beim schnellen Abbremsen und Umkehren der Hand treten jedoch größere Beschleunigungen auf, sodass alle Werte darüber auf den Maximalwert abgeschnitten werden (*Clipping*). Lösen lässt sich das, indem man den Messbereich im Register `XYZ_DATA_CFG` auf $\pm 4\,g$ oder $\pm 8\,g$ vergrößert (auf Kosten der Auflösung pro LSB). [**TODO:** Konkreten Plateau-Wert aus euren Daten angeben (z. B. ca. $\pm 2\,g$).]

**(c) Abweichung von der tatsächlichen Bewegung:** Neben dem Clipping ist vor allem die Datenrate `ODR_1` = 1,56 Hz viel zu niedrig. Eine schnelle Auf- und Abbewegung enthält Frequenzanteile deutlich über 1 Hz; nach dem Abtasttheorem müsste mit mehr als dem Doppelten der höchsten Signalfrequenz abgetastet werden. Bei rund 1,5 Werten pro Sekunde wird pro Bewegung höchstens ein bis zwei Punkte erfasst, kurze Spitzen fallen zwischen die Abtastzeitpunkte und es kommt zu Aliasing. Zusätzlich misst der Sensor immer auch die Erdbeschleunigung mit, und bereits kleine Verkippungen der Hand verteilen diese auf mehrere Achsen. [**TODO:** Mit euren Beobachtungen aus Abb. 2 abgleichen.]

## 4.2 ANPASSUNG DER SENSORKONFIGURATION (AUFGABE 5)

Laut Datenblatt des MMA8452Q [1] sind zwei Register relevant (Tab. 1):

Tabelle 1: Relevante Register des MMA8452Q und mögliche Parameterwerte.

| Register / Parameter | mögliche Werte | Datenblatt |
|---|---|---|
| `CTRL_REG1` (0x2A), Bits `DR[2:0]` | 800, 400, 200, 100, 50, 12,5, 6,25, 1,56 Hz | S. [**TODO:** ?] |
| `XYZ_DATA_CFG` (0x0E), Bits `FS[1:0]` | $\pm 2\,g$, $\pm 4\,g$, $\pm 8\,g$ (11 = reserviert) | S. [**TODO:** ?] |

In der SparkFun-Bibliothek entsprechen diese den Funktionen `setDataRate()` (`ODR_800` … `ODR_1`) und `setScale()` (`SCALE_2G`, `SCALE_4G`, `SCALE_8G`). Der Code wurde wie folgt geändert:

```
accel.setScale(SCALE_8G); // vorher SCALE_2G -> kein Clipping mehr
accel.setDataRate(ODR_100); // vorher ODR_1 (1,56 Hz)
```

[**TODO:** Prüfen, ob ihr genau diese Werte verwendet habt, sonst anpassen.]

**Begründung:** Menschliche Handbewegungen liegen im Wesentlichen unter ca. 10–20 Hz, eine Abtastrate von 100 Hz erfüllt damit das Abtasttheorem mit ausreichender Reserve. Höhere Raten sind über die serielle Verbindung nicht sinnvoll: Eine Zeile (`X\tY\tZ\tZeit\r\n`) umfasst ca. 26 Zeichen ≈ 260 Bit; bei 57 600 Baud sind damit nur ca. 220 Zeilen/s übertragbar, 400 oder 800 Hz würden also Daten verlieren. Der Messbereich $\pm 8\,g$ verhindert die Sättigung aus Aufgabe 4; die Auflösung von $1/256\,g \approx 0{,}04\,\mathrm{m/s^2}$ ist für Bewegungen ausreichend.

Abbildung 3 zeigt die wiederholte Messung.

**PLATZHALTER – Abbildung noch einfügen**

Aufgabe 5: 4 s der Daten mit geändertem Code (gleiche Achsenskalierung wie Abb. 2, damit der Vergleich leicht fällt).

Abbildung 3: Beschleunigung während der Auf- und Abbewegung mit angepasstem Messbereich und Datenrate (Ausschnitt 4 s).

**Vergleich:** [**TODO:** Gemeinsamkeiten/Unterschiede zu Abb. 2 beschreiben, z. B.:] Gemeinsam ist beiden Messungen die periodische Struktur mit etwa einer Bewegung pro Sekunde sowie der Gleichanteil von ca. $1\,g$ in der vertikalen Achse (Erdbeschleunigung). Im Unterschied zu Abb. 2 sind die Peaks nun nicht mehr abgeschnitten, sondern zeigen die tatsächlichen Maximalwerte von [**TODO:** ?] $g$, und der Signalverlauf ist durch die rund 64-fach höhere Datenrate glatt und zeitlich fein aufgelöst. Dafür ist das Rauschen sichtbar, weil nun auch kleine, schnelle Schwankungen erfasst werden.

## 4.3 Zeitstempel und statische Messung in sechs Orientierungen (Aufgabe 6)

Jede Messung wird direkt auf dem Mikrocontroller mit einem Zeitstempel versehen (`millis()` [**TODO:** bzw. `micros()` – verwendete Funktion angeben]), nicht über die Zeitstempel des seriellen Monitors. Aus den Zeitdifferenzen $\Delta t_i = t_{i+1} - t_i$ werden in Python die mittlere Abtastfrequenz und deren Streuung berechnet (siehe Abschnitt 4.5).

Der Sensor wurde nacheinander in alle sechs Orientierungen gebracht ($\pm x$, $\pm y$, $\pm z$ jeweils nach oben) und jeweils 10 s ruhig gehalten.

**(a)** Abbildung 4 zeigt die gesamte Messung.

**PLATZHALTER – Abbildung noch einfügen**

Aufgabe 6(a): gesamte Messung der 6 Orientierungen (ca. 60 s), x/y/z über $t$/s, Werte in $g$; Abschnitte ggf. beschriften (+x oben, −x oben, ...).

Abbildung 4: Statische Messung in allen sechs Orientierungen des Sensors (je 10 s).

**(b) Plausibilität:** In Ruhe misst der Sensor nur die Erdbeschleunigung. Zeigt eine Achse nach oben, sollte sie ca. $+1\,g$, nach unten ca. $-1\,g$ und die beiden anderen Achsen ca. $0\,g$ anzeigen. (Ein Beschleunigungssensor misst die Reaktionskraft der Unterlage, daher zeigt die nach oben gerichtete Achse $+1\,g$.) [**TODO:** Gemessene Mittelwerte je Orientierung angeben, z. B. als kleine Tabelle, und Abweichungen von $\pm 1\,g$ diskutieren (Offset, Verkippung von Hand, Empfindlichkeitstoleranz laut Datenblatt).]

## 4.4 FILTERUNG DER DATEN (AUFGABE 7)

Die Daten aus Aufgabe 6 wurden mit einem digitalen Butterworth-Tiefpass 4. Ordnung mit der Grenzfrequenz $f_g$ = [**TODO:** ?] Hz gefiltert. Damit das Signal zeitlich nicht verschoben wird, wurde die Funktion `filtfilt()` aus `scipy.signal` verwendet: Der Filter wird einmal vorwärts und einmal rückwärts angewandt, wodurch die Phasenverschiebung aufgehoben wird (Nullphasenfilter). Abbildung 5 vergleicht gefilterte und ungefilterte Daten.

**PLATZHALTER – Abbildung noch einfügen**

Aufgabe 7: ungefilterte (z. B. dünn, grau) vs. gefilterte Daten (durchgezogen) einer Achse bzw. aller Achsen; Ausschnitt so wählen, dass der Effekt gut sichtbar ist.

Abbildung 5: Vergleich der ungefilterten und mit einem Butterworth-Tiefpass gefilterten Beschleunigungsdaten.

**Effekt und Begründung:** [**TODO:** 1–2 Sätze an eure Ergebnisse anpassen:] Der Tiefpass entfernt das hochfrequente Sensor- und Quantisierungsrauschen, sodass die Plateaus der einzelnen Orientierungen als glatte Werte bei ca. $\pm 1\,g$ bzw. $0\,g$ erscheinen, während die Übergänge zwischen den Orientierungen erhalten bleiben. Ein Tiefpass wurde gewählt, weil die interessierende Information (Lage bzw. langsame Bewegung) im niedrigen Frequenzbereich liegt und das Rauschen breitbandig ist.

## 4.5 BESTIMMUNG DER ABTASTFREQUENZ (AUFGABE 8)

Aus den Zeitstempeln der gesammelten Daten wurde die Abtastfrequenz berechnet:

$$\bar{f}_s = \frac{1}{\overline{\Delta t}}, \qquad \overline{\Delta t} = \frac{1}{N-1}\sum_{i=1}^{N-1}(t_{i+1} - t_i), \qquad \sigma_{\Delta t} = \sqrt{\frac{1}{N-2}\sum_{i=1}^{N-1}\left(\Delta t_i - \overline{\Delta t}\right)^2}. \tag{4}$$

Die Streuung der Frequenz wurde [**TODO:** direkt aus $f_i = 1/\Delta t_i$ bzw. über Fehlerfortpflanzung $\sigma_f \approx \sigma_{\Delta t}/\overline{\Delta t}^{\,2}$ – Methode angeben] bestimmt. Abbildung 6 zeigt die Verteilung der Abtastintervalle, Tab. 2 die Ergebnisse.

**PLATZHALTER – Abbildung noch einfügen**

Aufgabe 8: Histogramm der Abtastintervalle $\Delta t$/ms (oder $\Delta t$ über Sample-Index), Mittelwert als vertikale Linie einzeichnen.

Abbildung 6: Verteilung der Abtastintervalle des stationären Messsystems.

Tabelle 2: Abtastfrequenz des stationären (USB) und des mobilen (DataLogger) Messsystems.

| Messsystem | $\overline{\Delta t}$ / ms | $\bar{f}_s$ / Hz | Standardabweichung $\sigma_f$ / Hz |
|---|---|---|---|
| stationär (USB, Aufgabe 8) | [**TODO:** ?] | [**TODO:** ?] | [**TODO:** ?] |
| mobil (DataLogger, Aufgabe 9) | [**TODO:** ?] | [**TODO:** ?] | [**TODO:** ?] |

[**TODO:** 1–2 Sätze Interpretation: Stimmt $\bar{f}_s$ mit der eingestellten ODR (100 Hz) überein? Bei `millis()` ist die Auflösung nur 1 ms, daher springen $\Delta t$ z. B. zwischen 9, 10 und 11 ms.]

### 4.6 MOBILES MESSSYSTEM (AUFGABE 9)

Für diese Messung wurde das mobile Messsystem mit DataLogger und Batterie verwendet. Mit dem Code `Lab1Code2` wurden ca. 10 s aufgezeichnet und die Abtastfrequenz analog zu Abschnitt 4.5 bestimmt (Tab. 2). [**TODO:** Kurz beschreiben: welcher DataLogger, wie die Daten gespeichert wurden (SD-Karte), Ergebnisse eintragen.]

**Diskussion:** [**TODO:** An eure Messwerte anpassen, z. B.:] Die mittlere Abtastfrequenz des mobilen Systems unterscheidet sich vom stationären Wert und streut stärker. Ursache ist, dass der Mikrocontroller die Daten nicht mehr nur seriell ausgibt, sondern an den DataLogger übergibt, der sie auf eine SD-Karte schreibt. Das Schreiben erfolgt blockweise und benötigt unregelmäßig mehrere Millisekunden, in denen keine neuen Werte gelesen werden. Zudem unterscheiden sich der verwendete Code (`Lab1Code2`, z. B. andere Datenrate oder Ausgabeformat), die Übertragungsgeschwindigkeit zum Logger und die Taktquelle. Die eingestellte Sensordatenrate stellt daher nur eine Obergrenze dar; die tatsächliche Abtastfrequenz wird von der langsamsten Stelle der Signalkette bestimmt.

# 5 zusammenfassung

[**TODO:** 3–5 Sätze Fazit, z. B.:] Das Messsystem aus Sensor, RedBoard und PC wurde aufgebaut und in Betrieb genommen. Mit der Standardkonfiguration ($\pm 2\,g$, 1,56 Hz) bildeten die Daten die Bewegung aufgrund von Sättigung und zu geringer Abtastrate nicht korrekt ab. Durch die Anpassung von Messbereich und Datenrate im Sensorregister konnte die Bewegung korrekt erfasst werden. Die statische Messung bestätigte die Erdbeschleunigung von ca. $1\,g$ in jeder Orientierung. Die tatsächlich erreichte Abtastfrequenz betrug [**TODO:** ?] Hz (stationär) bzw. [**TODO:** ?] Hz (mobil).

# literatur.

[1] NXP Semiconductors, “MMA8452Q, 3-axis, 12-bit/8-bit digital accelerometer,” Datenblatt, Rev. [**TODO:** ?]. [**TODO:** Version/Link des verwendeten Datenblatts eintragen.]

[2] SparkFun Electronics, “SparkFun RedBoard Qwiic – Hookup Guide,” online, [**TODO:** URL und Abrufdatum].

[3] P. Virtanen *et al.*, “SciPy 1.0: Fundamental algorithms for scientific computing in Python,” *Nature Methods*, vol. 17, pp. 261–272, 2020.

[4] [**TODO:** Vorlesungsunterlagen Biosignalverarbeitung (A. Morillo), MCI, 2026 – falls zitiert.]

# A anhang: abgegebene dateien (aufgabe 10)

Die Abgabe `Lab1_Gruppe_X.zip` enthält:

- `Bericht_1_Gruppe_X.pdf` – dieser Bericht,
- `Lab1Code1.ino` – modifizierter Arduino-Code (Aufgabe 5),
- `MainCode1.ipynb` – Python-Auswertung (Aufgaben 4a, 5, 6a, 7, 8, 9), erzeugt alle Abbildungen dieses Berichts,
- alle Datensätze (`.txt`), die für `MainCode1.ipynb` benötigt werden,
- ggf. selbstgeschriebene Python-Module. [**TODO:** Liste an tatsächliche Dateinamen anpassen.]